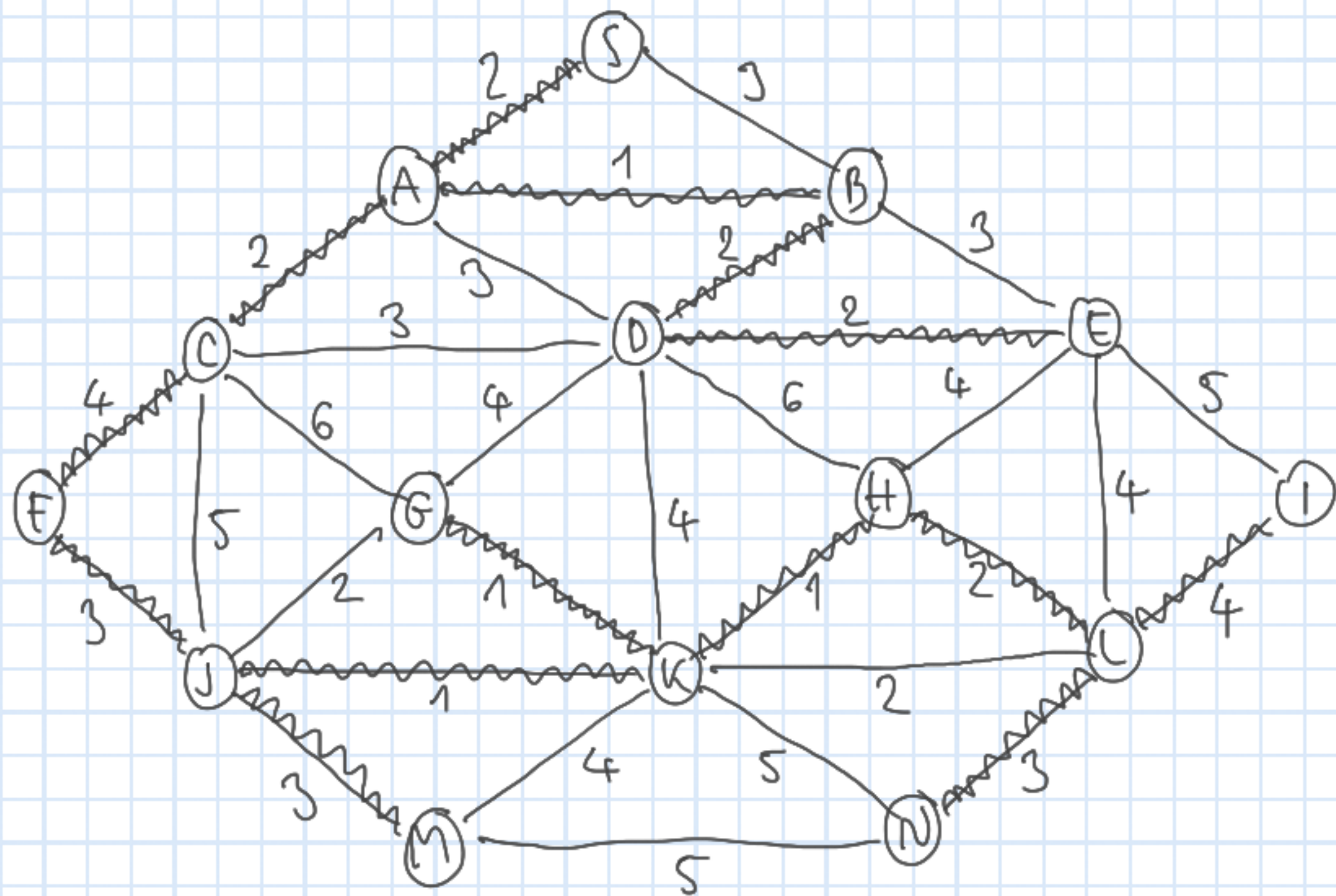


November 19, 2021

Example (cont'd)



Heap

~~A~~, SB

SB, ~~A~~, AC, AD

SB, ~~A~~, AD, BD, BE

SB, AD, ~~B~~, BE, CD, CF, CG, CJ

SB, AD, BE, CD, CF, CG, CJ,

~~DE~~, DG, DH, DK

~~SB~~, ~~AD~~, ~~BE~~, ~~DE~~, ~~DE~~, CG, CJ

DG, DH, DK, EH, EI, EL

CG, CJ, DG, DH, DK, EH, EI, EL,

~~FJ~~

Heap (cont'd)

CG, CJ, DG, DH, DK, EH, EI, EL, JG, ~~K~~, JM

CG, CJ, DG, DH, DK, EH, EI, EL, JG, JM, ~~K~~, KH, KL, KM, KN

CG, CJ, DG, DH, DK, EH, EI, EL, JG, JM, ~~KH~~, KL, KM, KN

CG, CJ, DG, DH, DK, EH, EI, EL, ~~K~~, JM, KL, KM, KN, ~~K~~

CG, CJ, DG, DH, DK, EH, EI, EL, JM, ~~K~~, KM, KN, LI, ~~K~~

CG, CJ, DG, DH, DK, EH, EI, EL, ~~JM~~, KM, KN, LI, MN

CG, CJ, ~~DG~~, DH, ~~DK~~, EH, EI, ~~K~~, ~~JM~~, KN, ~~K~~, MN

Cost of Prim's algorithm

We can check whether the edge obtained by a MINDELETE is an edge between X and $V \setminus X$ in constant time, if we have one bit information for each vertex showing that the vertex is in X or not.

After we add a vertex to X we can insert the edges adjacent to this vertex to the heap in $O(|E|)$ time altogether since the total length of the adjacency lists is $O(|E|)$ [$2|E|$, in fact]

The number of items in the heap is never bigger than $|E|$, hence the heap operations take $O(\log |E|)$ time

We have at most $|E|$ INSERTs and $|E|$ MINDELETEs

Thus the total cost is $O(|E| \log |E|)$

Kruskal algorithm

Another MST algorithm with complexity $O(|E| \log |E|)$

We start building the MST at different places, and we join the components subsequently

Initially we have $|V|$ one-vertex components, and finally we have 1 $|V|$ -vertex component.

In each step we take the min weight edge (from the remaining ones) and

- if the edge joins two different components (i.e., it doesn't produce a cycle with the edges selected previously), then we join these two components by this edge
- if the edge joins two vertices of the same component, we drop this edge (in this case the edge would produce a cycle with the edges selected previously)

Algorithm

We will store the edges of the MST in a set F .

We will use another set H containing the edges that we will have to consider later on.

Initially $F = \emptyset$ and $H = E$

Kruskal (G)

$F := \emptyset$, $H := E$

while $H \neq \emptyset$ do

 Remove a min weight edge (u, v) from H

 if $F \cup \{(u, v)\}$ doesn't contain a cycle (i.e., u and v belong to different components) then $F := F \cup \{(u, v)\}$

Implementation

To implement Kruskal's algorithm efficiently we will need a new Data Structure

→ Union - Find

This will help us to decide whether (u, v) connects different components or not.

Union - Find

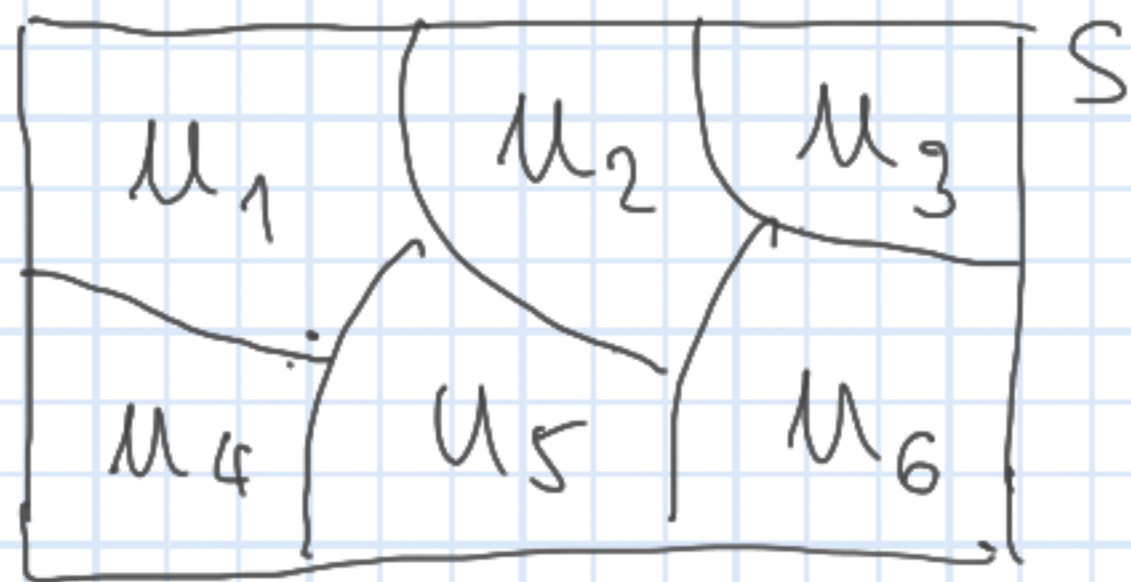
We have a base set (universe) S and the data structure stores a partition of S

$$S \rightarrow \{u_1, u_2, \dots, u_k\}$$

$$\textcircled{1} \quad u_1, u_2, \dots, u_k \subseteq S$$

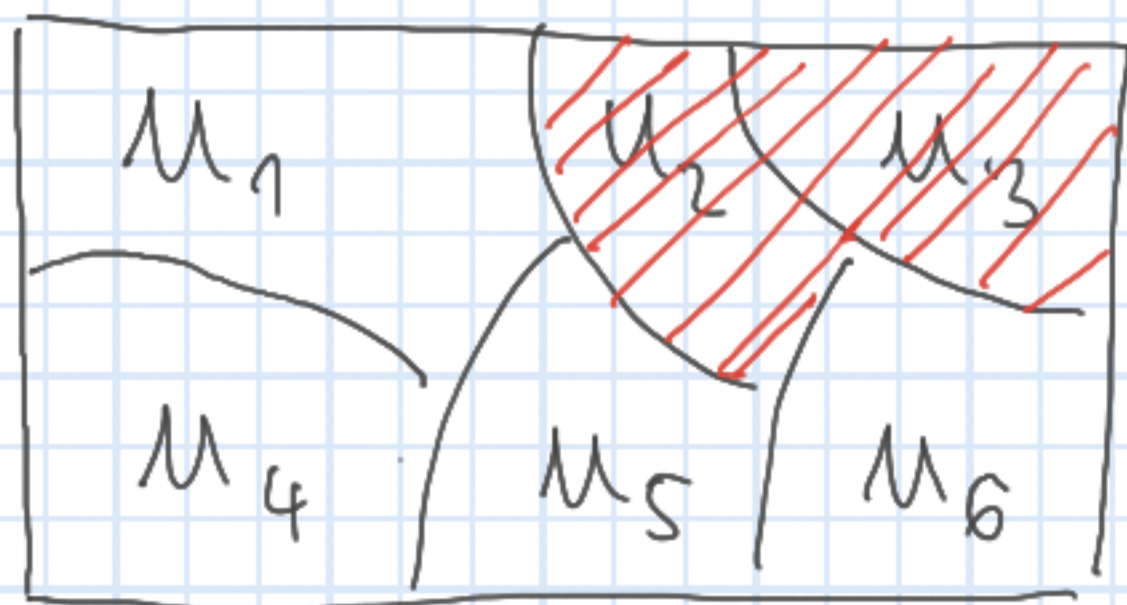
$$\textcircled{2} \quad u_1 \cup u_2 \cup \dots \cup u_k = S$$

$$\textcircled{3} \quad u_i \cap u_j = \emptyset \quad \text{if } i \neq j$$



Operations

- Union $(u_i, u_j) =$
 $(\{u_1, u_2, \dots, u_k\} \setminus \{u_i, u_j\}) \cup \{u_i \cup u_j\}$



after taking union
we have a 5-member
partition

- Find (x) where $x \in S$
this is the member of the partition
containing x

